

Problem 1 & 2 Explanation

Nama: Aldino Fairuz

Kelompok: Obsidian

Problem 1:

Hasil Pseudocode:

```
BEGIN

INPUT N // Jumlah astronot
INPUT K // Nilai langkah eliminasi (K) awal


// Validasi input awal
IF N <= 0 THEN
    STOP / EXIT
END IF


IF K < 2 THEN
    K = 2
END IF


// Inisialisasi daftar astronot
CREATE LIST astro
FOR i = 1 TO N DO
    APPEND i TO astro
END FOR


CREATE LIST gugur // Untuk menyimpan urutan yang tereliminasi
SET pos = 0 // Indeks posisi saat ini


PRINT "=== Log Eliminasi ==="

// Lakukan eliminasi sampai tersisa 1 orang
```

```

WHILE LENGTH(astro) > 1 DO
    // Hitung indeks yang akan dihapus
    SET hapus = (pos + K - 1) MODULO LENGTH(astro)
    SET id = astro[hapus]

    APPEND id TO gugur
    PRINT "Astronot " + id + " gugur (K awal=" + K + ")."

    // Hapus astronot dari daftar dan perbarui posisi
    REMOVE ITEM AT INDEX hapus FROM astro
    SET pos = hapus

    // Modifikasi nilai K berdasarkan ID yang baru saja gugur
    IF id MODULO 2 == 0 THEN
        // Jika ID genap, tambah K dengan 2
        K = K + 2
    ELSE
        // Jika ID ganjil, kurangi K dengan 1
        K = K - 1
        IF K < 2 THEN
            K = 2
        END IF
    END IF

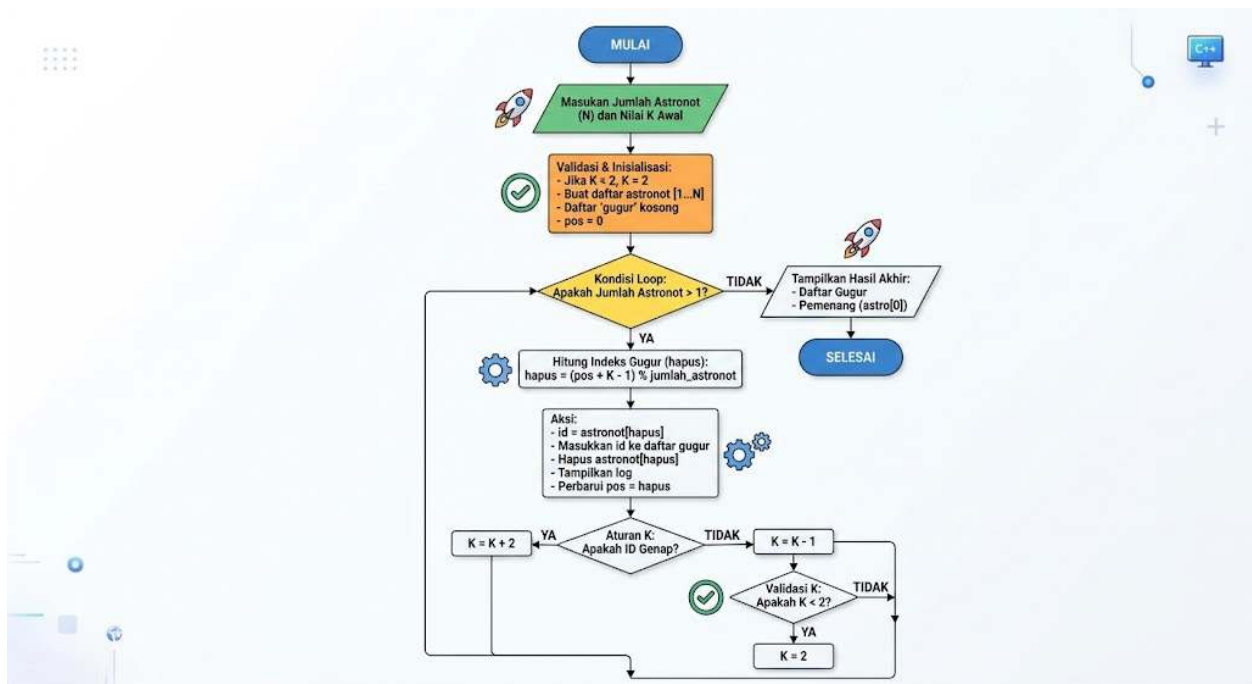
    PRINT "K baru=" + K
END WHILE

PRINT "=== Hasil Akhir ==="
PRINT "Urutan gugur: " + JOIN(gugur, ", ")
PRINT "Astronot bertahan: " + astro[0]

END

```

Hasil flowchart:



Penjelasan singkat:

Tujuan utama dari kode tersebut adalah mensimulasikan proses eliminasi sekelompok astronot untuk mencari satu orang terakhir yang bertahan hidup, yang diadaptasi dari teka-teki logika klasik Josephus Problem. Program ini meminta pengguna untuk menentukan jumlah total astronot (N) dan jarak lompatan awal (K) untuk memulai simulasi. Melalui proses ini, sistem tidak hanya mencari siapa pemenang akhirnya, tetapi juga melacak dan menampilkan secara detail rekam jejak urutan gugurnya para astronot dari langkah pertama hingga akhir.

Secara logika, program menggunakan struktur perulangan yang akan terus mengeliminasi peserta selama jumlah astronot di dalam daftar masih lebih dari satu orang. Penentuan posisi astronot yang gugur dihitung menggunakan aritmatika modulo $(pos + K - 1) \% sisa_astronot$, yang berfungsi untuk memastikan penghitungan terus berputar secara melingkar meskipun sudah mencapai ujung antrian. Keunikan utama dari logika kode ini terletak pada aturan nilai K yang dinamis; jika astronot yang tereliminasi memiliki nomor identitas genap, maka nilai lompatan (K) pada putaran berikutnya akan ditambah dua, sedangkan jika ganjil, nilai K dikurangi satu (dengan memastikan nilai minimum K adalah 2). Hal ini membuat pola eliminasi terus berubah secara adaptif di setiap putarannya.

Problem 2:

Hasil pseudocode:

```
FUNCTION encrypt(message)
```

```
    INITIALIZE result = ""
```

```
    INITIALIZE prev_val = 0
```

```
    FOR i = 0 TO length(message) - 1 DO
```

```
        ch = TO_UPPERCASE(message[i])
```

```
        IF ch is in the range 'A' to 'Z' THEN
```

```
            // Calculate numeric value of the letter (A=1, B=2, ..., Z=26)
```

```
            val = ASCII_VALUE(ch) - ASCII_VALUE('A') + 1
```

```
            IF i == 0 THEN
```

```
                new_val = val
```

```
            ELSE
```

```
                new_val = ((val - 1 + prev_val) MODULO 26) + 1
```

```
            END IF
```

```
            // Convert numeric value back to a character
```

```
            encrypted_char = TO_CHARACTER(ASCII_VALUE('A') + new_val - 1)
```

```
            result = result + encrypted_char
```

```
            // Store original character value for the next iteration
```

```
            prev_val = val
```

```
        ELSE
```

```
            // If not an alphabet letter, append original character unencrypted
```

```
            result = result + message[i]
```

```
            prev_val = 0
```

```
        END IF
```

END FOR

RETURN result

END FUNCTION

MAIN PROGRAM

PRINT "Enter text: "

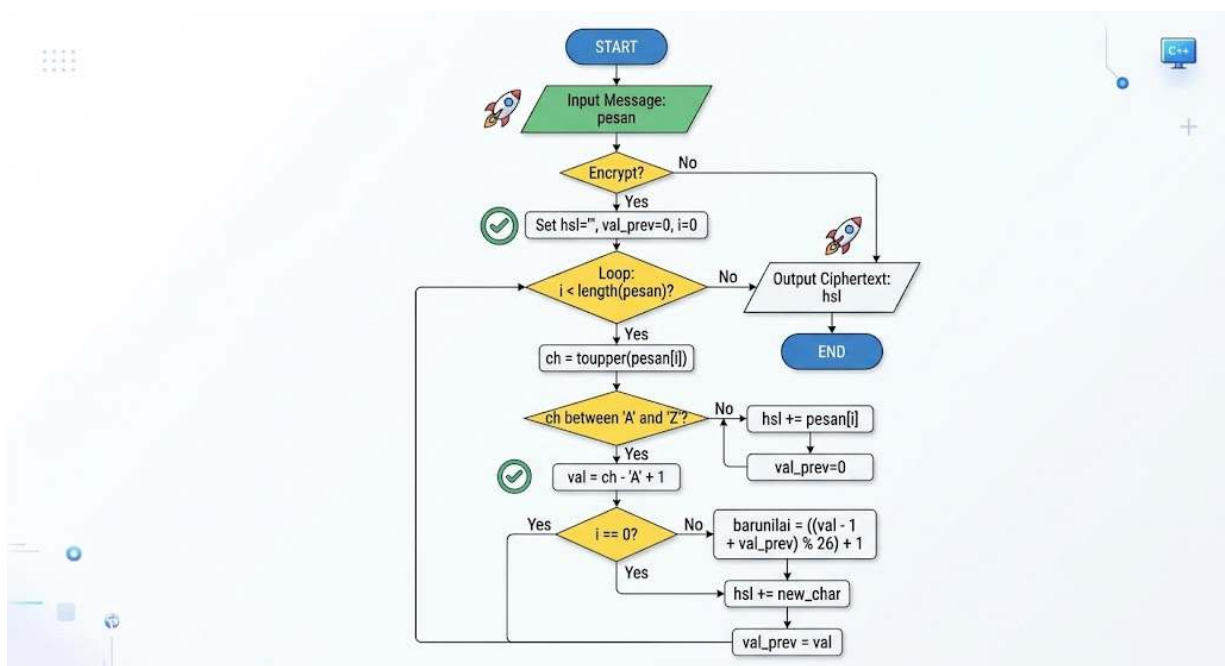
READ message

cipher_text = CALL encrypt(message)

PRINT "Cipher result: " + cipher_text

END MAIN

Hasil Flowchart:



Penjelasan singkat:

Program ini bekerja dengan cara membaca setiap karakter dari pesan teks satu per satu dan mengubahnya menjadi huruf kapital. Logika utama memisahkan perlakuan antara karakter alfabet dan non-alfabet; jika karakter bukanlah huruf dari A hingga Z, program akan langsung menambahkannya ke teks hasil sandi tanpa ada enkripsi atau perubahan. Pada kondisi karakter non-alfabet ini, nilai penyimpanan huruf sebelumnya

(val_prev) juga direset menjadi nol, sehingga memastikan spasi, angka, atau tanda baca tidak memberikan efek pergeseran matematis pada huruf yang muncul setelahnya.

Jika karakter yang diproses adalah huruf alfabet, program menghitung nilai posisi huruf tersebut dari angka 1 hingga 26. Untuk huruf paling awal pada iterasi pertama, karakter akan langsung diproses tanpa pergeseran nilai. Namun, untuk huruf-huruf selanjutnya, program menciptakan karakter sandi baru dengan cara menambahkan nilai posisi huruf saat ini dengan nilai asli dari huruf yang mendahuluinya, lalu menggunakan operasi modulo 26 agar perhitungannya terus berputar dalam rentang alfabet yang valid. Setelah karakter baru didapat, program akan memperbarui memori sistem dengan menyimpan nilai huruf asli saat ini untuk digunakan sebagai angka pergeseran pada huruf berikutnya.